

Barbex: Sistema Web para la Gestión Integral de Citas, Servicios y Pagos en una Barbería

Kenneth Ramírez, Jeremy Chan, Derek Rayo

Escuela de Computación, Universidad Fidélitas

San José, Costa Rica

kramirez40173@ufide.ac.cr, jchan90556@ufide.ac.cr, drayo60253@ufide.ac.cr

Resumen—Las barberías pequeñas e independientes suelen administrar sus citas mediante agendas físicas, llamadas telefónicas o mensajería instantánea, lo que genera choques de horario, inasistencias y ausencia de control sobre adelantos y pagos. Este artículo documenta el desarrollo de Barbex, una aplicación web transaccional construida con Java 21, Spring Boot 3, Hibernate/JPA, MySQL, Thymeleaf y Bootstrap 5, orientada a la gestión integral de citas, servicios y pagos de una barbería. El sistema permite a los clientes agendar citas seleccionando servicios y horarios disponibles calculados automáticamente, y a los barberos administrar su horario semanal, excepciones puntuales, catálogo de servicios y solicitudes de cambio o cancelación. Se describen la arquitectura por capas, el modelo de datos, la lógica de validación de disponibilidad y el flujo de estados de una cita, así como las funcionalidades implementadas. Los resultados muestran un sistema funcional que reduce los conflictos de agenda y centraliza la operación de la barbería, sentando una base extensible para trabajo futuro como pagos en línea y notificaciones automáticas.

Palabras clave—gestión de citas, aplicación web, Spring Boot, sistema transaccional, patrón MVC, barbería.

Abstract—Small, independent barbershops often manage appointments through paper agendas, phone calls, or instant messaging, which leads to scheduling conflicts, no-shows, and a lack of control over deposits and payments. This paper documents the development of Barbex, a transactional web application built with Java 21, Spring Boot 3, Hibernate/JPA, MySQL, Thymeleaf, and Bootstrap 5, aimed at the comprehensive management of appointments, services, and payments in a barbershop. The system allows clients to book appointments by selecting services and automatically computed available time slots, while barbers manage their weekly schedule, one-time exceptions, service catalog, and change or cancellation requests. The layered architecture, data model, availability-validation logic, and appointment state flow are described, together with the implemented functionalities. Results show a working system that reduces scheduling conflicts and centralizes barbershop operations, providing an extensible foundation for future work such as online payments and automated notifications.

Keywords—appointment management, web application, Spring Boot, transactional system, MVC pattern, barbershop.

I. INTRODUCCIÓN

Las barberías independientes constituyen un modelo de negocio con alta rotación de clientes y una fuerte dependencia de la puntualidad en la prestación de sus servicios. En la práctica, gran parte de estos negocios continúa gestionando sus citas de forma manual, mediante cuadernos, llamadas telefónicas o aplicaciones de mensajería como WhatsApp. Esta forma de operar es propensa a errores humanos: choques de horario entre dos clientes, olvido de citas, ausencia de un historial ordenado de servicios prestados y nula trazabilidad sobre los adelantos o pagos parciales que el negocio solicita para asegurar la asistencia del cliente.

La falta de una herramienta digital especializada genera, además, dificultades para que el barbero controle su disponibilidad real cuando debe ausentarse en un día específico o cambiar su horario habitual, así como para que el cliente conozca de forma inmediata los espacios disponibles antes de solicitar una cita.

Ante esta problemática, el presente proyecto desarrolla Barbex, un sistema web transaccional que centraliza la gestión de citas, servicios y pagos de una barbería. El objetivo general es proveer una plataforma que permita a los clientes agendar citas de forma autónoma, respetando la disponibilidad real del barbero, y a los barberos administrar su agenda, catálogo de servicios y horario laboral desde una única interfaz. Como objetivos específicos se plantean: (1) automatizar el cálculo de horarios disponibles a partir del horario semanal y las excepciones puntuales del barbero; (2) implementar un flujo de confirmación de citas basado en el pago de un adelanto cuando el servicio lo requiera; (3)

permitir la gestión de solicitudes de cambio o cancelación de citas por parte del cliente, sujetas a aprobación del barbero; y (4) sentar una arquitectura extensible en capas que facilite el mantenimiento futuro del sistema.

El alcance del sistema contempla dos roles de usuario —barbero y cliente— y soporta múltiples barberías de forma independiente, cada una identificada mediante un código de acceso único que los clientes utilizan para vincularse a su barbería durante el registro.

II. MARCO TEÓRICO / REVISIÓN DE LITERATURA

El desarrollo de Barbex se apoya en un conjunto de tecnologías consolidadas del ecosistema Java para aplicaciones web transaccionales, seleccionadas por su madurez, documentación y facilidad de integración.

A. Spring Boot y el patrón MVC

Spring Boot 3.x es un framework que simplifica la configuración de aplicaciones Java basadas en el contenedor Spring, proporcionando arranque automático, inyección de dependencias y un servidor embebido. Su uso favorece la organización del código siguiendo el patrón Modelo-Vista-Controlador (MVC), separando la lógica de negocio (servicios), el acceso a datos (repositorios) y la presentación (controladores y plantillas).

B. Persistencia con Hibernate y Spring Data JPA

Hibernate, a través de la especificación Jakarta Persistence (JPA) e integrado mediante Spring Data JPA, permite mapear las entidades del dominio a tablas relacionales sin necesidad de escribir sentencias SQL

manuales para las operaciones básicas. Los repositorios de Spring Data JPA exponen métodos derivados a partir de convenciones de nombres, lo cual reduce el código repetitivo en el acceso a datos.

C. Base de datos MySQL

MySQL 8.0 se utiliza como motor de persistencia relacional, dado su soporte robusto para transacciones, restricciones de integridad referencial y tipos de datos temporales, necesarios para modelar fechas, horas y montos monetarios con precisión decimal.

D. Motor de plantillas Thymeleaf

Thymeleaf es un motor de plantillas del lado del servidor que permite generar vistas HTML dinámicas integradas de forma natural con Spring Boot. Su sintaxis basada en atributos (th:text, th:each, th:if) facilita la reutilización de fragmentos comunes, como la plantilla base que integra la barra de navegación, el pie de página y el sistema de notificaciones utilizado en Barbex.

E. Bootstrap y control de versiones

Bootstrap 5 provee un sistema de diseño responsivo basado en componentes predefinidos (formularios, tablas, tarjetas, modales) que agiliza la construcción de una interfaz consistente sin necesidad de escribir CSS extenso. Paralelamente, Git y GitHub se emplean como sistema de control de versiones distribuido, permitiendo el trabajo colaborativo del equipo mediante ramas e integración del código en un repositorio remoto compartido.

F. Justificación de la selección tecnológica

Durante la etapa de planificación se consideraron alternativas como Node.js con Express o Django (Python) para la capa de backend. Se optó por Spring Boot debido a la experiencia previa del equipo con el lenguaje Java, su tipado estático —que reduce errores en tiempo de compilación al modelar entidades financieras y de horario— y la integración nativa que ofrece con Hibernate/JPA y Thymeleaf dentro de un mismo ecosistema. La Tabla I resume las alternativas evaluadas.

Aspecto	Spring Boot (elegido)	Node.js/Express
Tipado	Estático (Java)	Dinámico (JavaScript)
ORM integrado	Spring Data JPA/Hibernate	Requiere ORM externo
Curva de aprendizaje	Media, experiencia previa del equipo	Baja, pero sin experiencia previa
Vistas del lado del servidor	Thymeleaf nativo	Requiere motor adicional

TABLA I. COMPARACIÓN DE ALTERNATIVAS TECNOLÓGICAS

III. METODOLOGÍA

El desarrollo del proyecto se organizó de forma iterativa, dividiendo el sistema en módulos funcionales —autenticación, gestión de citas del cliente, gestión de citas y horarios del barbero— que fueron asignados a los integrantes del equipo y desarrollados en paralelo sobre ramas independientes del repositorio Git, integrándose posteriormente sobre la rama principal.

El proceso de trabajo siguió, en cada módulo, las siguientes etapas: (1) definición del modelo de datos y sus relaciones mediante entidades JPA; (2) implementación de la capa de repositorios y servicios con la lógica de negocio y sus validaciones; (3) construcción de los controladores Spring MVC encargados de recibir las peticiones HTTP y coordinar la interacción entre las vistas y los servicios; (4) diseño de las plantillas Thymeleaf con Bootstrap para la interfaz de usuario; y (5) pruebas manuales por caso de uso, verificando reglas de negocio como el traslape de citas o la validez del horario laboral.

Como herramientas de apoyo se utilizaron Java 21 como lenguaje de programación, Maven (mediante el wrapper del proyecto) para la gestión de dependencias y construcción, Spring Boot 3.3.5 como framework principal, MySQL 8.0 como motor de base de datos, Thymeleaf 3.x para la generación de vistas, Bootstrap 5.3.3 e íconos de Bootstrap Icons para la interfaz, y Git/GitHub para el control de versiones y la integración del código del equipo.

IV. DESARROLLO

Barbex se estructura siguiendo una arquitectura en capas típica de una aplicación Spring Boot: entidades (entity), repositorios (repository), servicios (service), controladores (controller) y vistas (templates). Esta separación aísla la lógica de negocio de los detalles de persistencia y de presentación, facilitando el mantenimiento y las pruebas.

A. Modelo de datos

El dominio del sistema se modela mediante nueve entidades principales, resumidas en la Tabla II.

Entidad	Descripción
Usuario	Representa a barberos y clientes; almacena credenciales, rol (BARBERO/CLIENTE) y la barbería a la que pertenece.
Barberia	Negocio identificado por un código de acceso único; agrupa a sus usuarios.
Servicio	Catálogo de servicios ofrecidos por una barbería (precio, duración, adelanto).
BarberoServicio	Relación que indica qué servicios ofrece cada barbero.
Cita	Reserva entre cliente y barbero; incluye fecha, horario, montos y estado.
CitaServicio	Detalle de los servicios incluidos en una cita.
HorarioGeneral	Horario laboral semanal recurrente de un barbero por día.
ExcepcionHorario	Excepción puntual (día libre o jornada especial) sobre el horario general.
SolicitudCambioCita	Solicitud de edición o cancelación de una cita, sujeta a aprobación.

TABLA II. ENTIDADES PRINCIPALES DEL SISTEMA

La Tabla III detalla los campos principales de cada entidad del modelo relacional, incluyendo llaves primarias (PK), llaves foráneas (FK) y restricciones relevantes.

Entidad	Campos principales	Descripción
barberias	id (PK), nombre, direccion, telefono, codigo_acceso (UNIQUE), fecha_registro	Entidad principal; cada barbería registrada en la plataforma.
usuarios	id (PK), barberia_id (FK), nombre_completo, email	Unifica a barberos y

Entidad	Campos principales	Descripción
	(UNIQUE), password, telefono, rol (ENUM), activo	clientes del sistema.
servicios	id (PK), barberia_id (FK), nombre, descripcion, precio, duracion_minutos, monto_adelanto, activo	Servicios ofrecidos por cada barbería.
barbero_servicios	id (PK), barbero_id (FK), servicio_id (FK)	Asociativa M:N entre barberos y servicios.
citas	id (PK), barberia_id/barbero_id/cliente_id (FK), fecha, hora_inicio, hora_fin, monto_total, estado (ENUM), adelanto_pagado, monto_adelanto, comprobante_adelanto, motivo_cancelacion, fecha_creacion	Entidad transaccional central del sistema.
cita_servicios	id (PK), cita_id (FK), servicio_id (FK)	Asociativa M:N entre citas y servicios.
horarios_generales	id (PK), barbero_id (FK), dia_semana (ENUM), hora_inicio, hora_fin, libre, activo	Disponibilidad semanal recurrente del barbero.
excepciones_horario	id (PK), barbero_id (FK), fecha, hora_inicio, hora_fin, tipo (ENUM), motivo	Excepciones puntuales al horario habitual.
solicitudes_cambio_cita	id (PK), cita_id (FK), tipo (ENUM), nueva_fecha, nueva_hora_inicio, estado_solicitud (ENUM), motivo_rechazo, fecha_solicitud, fecha_respuesta	Solicitudes de edición o cancelación de una cita.

TABLA III. DICCIONARIO DE DATOS DEL MODELO RELACIONAL

Las relaciones entre estas entidades siguen, en su mayoría, cardinalidades uno-a-muchos: una barbería agrupa a varios usuarios y servicios; un barbero puede tener varias citas, varios horarios generales (uno por día) y varias excepciones; y una cita puede incluir varios servicios a través de la tabla intermedia CitaServicio. Esta normalización evita la duplicación de información y permite calcular montos totales y duraciones de forma consistente a partir de los servicios asociados a cada cita.

B. Autenticación y multi-barbería

El acceso al sistema se controla mediante sesión HTTP (HttpSession). Durante el registro, un usuario puede crear una nueva barbería —generándose un código de acceso único— o vincularse como cliente a una barbería existente seleccionándola de una lista. Las vistas Thymeleaf condicionan los elementos de navegación visibles según el rol del usuario autenticado.

C. Disponibilidad de horarios

La disponibilidad de un barbero se determina combinando dos fuentes: su horario general semanal (HorarioGeneral), definido por día de la semana, y sus excepciones puntuales (ExcepcionHorario), que pueden marcar un día como no laborable o redefinir el rango horario de una fecha específica, teniendo siempre prioridad sobre el horario general. A partir de esta información, el método calcularSlotsDisponibles()

recorre la jornada laboral en intervalos de treinta minutos, descartando los bloques que se traslapan con citas ya confirmadas o pendientes, y devuelve la lista de horas de inicio disponibles para la duración total de los servicios seleccionados.

D. Validación y ciclo de vida de una cita

Antes de persistir una cita, el método validarCita() verifica que la fecha no sea pasada, que la hora de fin sea posterior a la de inicio, que el horario solicitado esté dentro de la jornada laboral vigente del barbero y que no exista traslape con otra cita activa del mismo barbero. Cada cita transita por los estados definidos en el enumerado EstadoCita: PENDIENTE, CONFIRMADA, COMPLETADA, CANCELADA y RECHAZADA. Si los servicios seleccionados requieren un monto de adelanto, la cita se crea en estado PENDIENTE hasta que el barbero aprueba el comprobante; en caso contrario, se confirma automáticamente. El barbero puede además marcar una cita como completada, rechazarla o cancelarla directamente, mientras que el cliente puede editar la fecha/hora de una cita activa o generar una solicitud de cambio.

E. Solicitudes de cambio y citas afectadas

Cuando un cliente requiere modificar o cancelar una cita ya confirmada, el sistema registra una SolicitudCambioCita de tipo EDICION o CANCELACION, que el barbero puede aprobar o rechazar desde su panel de solicitudes. De forma complementaria, cuando el barbero modifica su horario general o registra una excepción que reduce su disponibilidad, el sistema identifica automáticamente las citas afectadas mediante los métodos obtenerCitasAfectadasPorCambioHorario(), obtenerCitasAfectadasPorDiaNoTrabajo() y obtenerCitasAfectadasPorExcepcion(), permitiéndole reagendarlas a un nuevo horario o cancelarlas desde una vista dedicada.

F. Controladores y módulos

La capa de presentación se organiza en cuatro controladores principales, descritos en la Tabla IV.

Controlador	Responsabilidad
AuthController	Login, registro de usuarios y cierre de sesión.
ClienteController	Panel de citas del cliente, creación, edición y solicitud de cancelación.
BarberoController	Agenda del barbero, gestión de servicios, horarios, excepciones y solicitudes.
HomeController	Redirección inicial según el rol del usuario autenticado.

TABLA IV. CONTROLADORES DEL SISTEMA

La interfaz se construye sobre una plantilla base común (layout/base.html) que integra la barra de navegación dependiente del rol, un sistema de notificaciones tipo toast para mensajes de éxito o error, y un modal de confirmación reutilizable para acciones críticas como la cancelación de una cita.

G. Seguridad y control de acceso

El control de acceso se implementa a nivel de sesión: cada petición a una ruta protegida verifica la existencia de un usuario autenticado y su rol antes de permitir el acceso a las

vistas y acciones correspondientes. Las rutas bajo /barbero quedan reservadas a usuarios con rol BARBERO y las rutas bajo /cliente a usuarios con rol CLIENTE, evitando que un cliente acceda a funciones administrativas como la aprobación de adelantos o la modificación del horario laboral.

H. Interfaz de usuario

Cada estado de una cita se representa visualmente mediante una insignia de color (badge) diferenciada — amarillo para PENDIENTE, verde para CONFIRMADA, azul para COMPLETADA, gris para CANCELADA y rojo para RECHAZADA— lo que permite al barbero identificar de un vistazo las citas que requieren su atención. Las citas vencidas, es decir, aquellas cuya fecha y hora ya transcurrieron sin haberse completado ni cancelado, se resaltan adicionalmente con un color de fondo distintivo en la tabla de citas.

I. Endpoints principales

La Tabla V resume las rutas HTTP más relevantes expuestas por los controladores del sistema.

Ruta	Método	Función
/login, /registro	GET/POST	Autenticación y alta de usuarios
/cliente/dashboard	GET	Historial de citas del cliente
/cliente/citas/nueva	GET/POST	Agendamiento de una nueva cita
/barbero/dashboard	GET	Agenda de citas del barbero
/barbero/citas/{id}/aprobar-adelanto	POST	Confirma el pago de un adelanto
/barbero/horarios	GET/POST	Horario semanal y excepciones
/barbero/horarios/afectadas	GET/POST	Reagendo o cancelación en lote
/barbero/solicitudes	GET/POST	Aprobación de solicitudes de cambio

TABLA V. ENDPOINTS PRINCIPALES DEL SISTEMA

II.API REST en postman

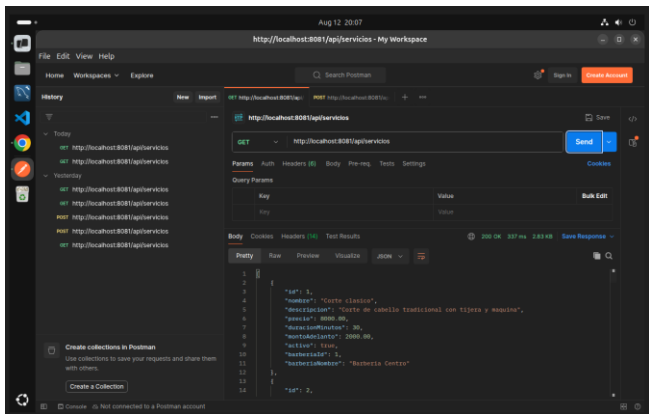


Fig. 1. Prueba del API en postman, obtener servicios

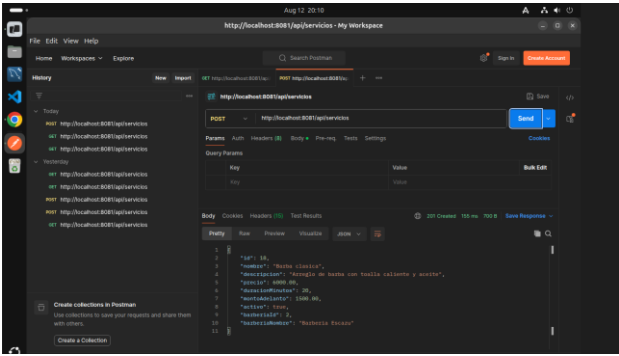


Fig. 2. Prueba del API en postman, crear un servicio

V. RESULTADOS

Como resultado del desarrollo se obtuvo un sistema funcional que cubre el flujo completo de gestión de una barbería, desde el registro de usuarios hasta el cierre del ciclo de vida de una cita. Entre las funcionalidades implementadas y verificadas se encuentran:

- Registro y autenticación diferenciada por rol, con creación o vinculación a una barbería mediante código de acceso.
- Gestión del catálogo de servicios por barbero (nombre, precio, duración y monto de adelanto).
- Configuración del horario semanal y de excepciones puntuales por parte del barbero.
- Agendamiento de citas por el cliente con cálculo automático de horarios disponibles según el barbero y los servicios seleccionados.
- Flujo de aprobación de adelantos y confirmación automática cuando el servicio no lo requiere.
- Edición y cancelación de citas, y solicitudes de cambio sujetas a aprobación del barbero.
- Panel de citas afectadas por cambios de horario, con opción de reagendar o cancelar en lote.

Las siguientes figuras ilustran las principales pantallas del sistema.

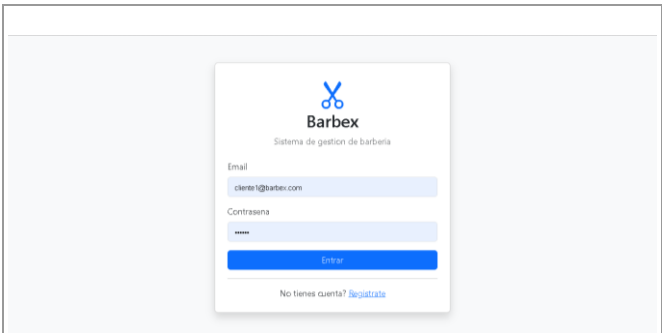


Fig. 3. Pantalla de inicio de sesión.

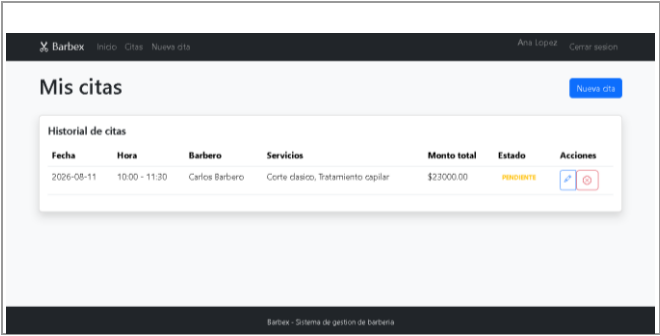


Fig. 4. Panel de citas del cliente.



Fig. 5. Formulario de agendamiento de una nueva cita.

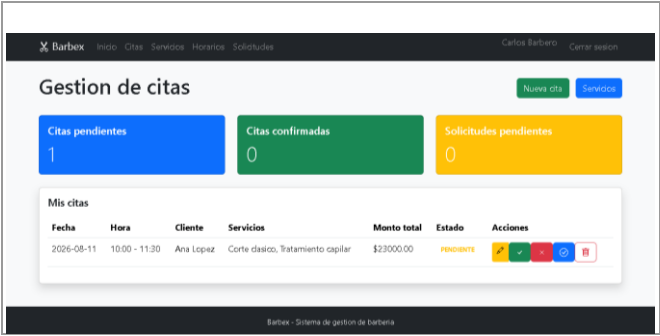


Fig. 6. Agenda de citas del barbero.

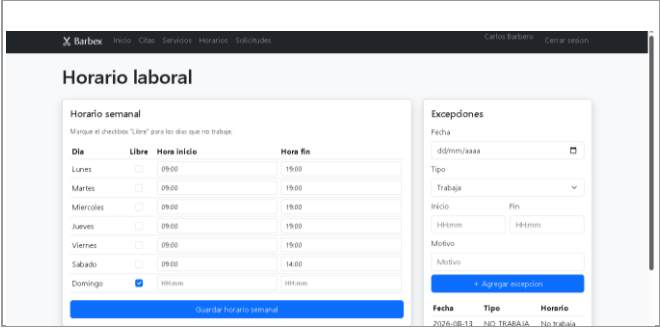


Fig. 7. Configuración de horarios y excepciones del barbero.

A. Pruebas funcionales

Las pruebas se realizaron de forma manual, caso de uso por caso de uso, verificando tanto el camino esperado como los escenarios de error de cada regla de negocio. La Tabla VI resume los casos de prueba principales aplicados sobre el módulo de citas.

Caso de prueba	Resultado esperado
Cita fuera del horario laboral	Rechazo con mensaje de horario inválido
Cita traslapada con otra existente	Rechazo con mensaje de conflicto
Cita en fecha pasada	Rechazo, no se permite agendar

Caso de prueba	Resultado esperado
Servicio con adelanto sin comprobante	Cita queda en estado RECHAZADA
Servicio sin adelanto requerido	Cita se confirma automáticamente
Barbero marca día como NO_TRABAJA	Citas existentes aparecen como afectadas

TABLA VI. CASOS DE PRUEBA DEL MÓDULO DE CITAS

En todos los casos evaluados el sistema respondió conforme a lo esperado, confirmando que las validaciones centralizadas en CitaService y HorarioService cubren adecuadamente las reglas de negocio definidas para el proyecto.

B. Discusión

El principal reto técnico enfrentado durante el desarrollo fue el cálculo correcto de los horarios disponibles cuando coexisten un horario general recurrente y excepciones puntuales que pueden ampliar, reducir o anular por completo la disponibilidad de un día concreto. Resolver este cálculo en un único método reutilizable, tanto para mostrar los horarios disponibles al cliente como para validar una cita al momento de guardarla, evitó duplicar la lógica de negocio y redujo el riesgo de inconsistencias entre lo que el cliente ve y lo que el sistema finalmente acepta.

VI. CONCLUSIONES

El desarrollo de Barbex permitió al equipo aplicar de forma práctica el patrón arquitectónico MVC sobre Spring Boot, así como profundizar en el modelado de relaciones complejas con JPA/Hibernate, particularmente en el diseño de la lógica de disponibilidad de horarios y en el manejo de los distintos estados de una cita. El uso de Git como sistema de control de versiones distribuido facilitó el trabajo colaborativo del equipo, permitiendo el desarrollo paralelo de los módulos de cliente y barbero.

Entre las limitaciones actuales del sistema se identifican la ausencia de una pasarela de pago real —el adelanto se valida mediante la revisión manual de un comprobante— y la falta de notificaciones proactivas hacia el cliente y el barbero, lo que representa la principal oportunidad de mejora a corto plazo.

Como trabajo futuro se identifican varias líneas de mejora: la integración de una pasarela de pagos en línea para el cobro de adelantos, en lugar de la validación manual de comprobantes; el envío de notificaciones automáticas por correo electrónico o mensajería para recordar citas próximas; la incorporación de reportes estadísticos de ingresos y ocupación por barbero; el desarrollo de pruebas automatizadas con JUnit para asegurar la estabilidad de las reglas de negocio; y el despliegue del sistema en un entorno en la nube para su disponibilidad en producción.

REFERENCIAS

[1] VMware, Inc., "Spring Boot Reference Documentation," Spring Framework, 2024. [En línea]. Disponible: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

[2] Red Hat, Inc., "Hibernate ORM User Guide," Hibernate, 2024. [En línea]. Disponible: <https://hibernate.org/orm/documentation/>

[3] Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle, 2024. [En línea]. Disponible: <https://dev.mysql.com/doc/refman/8.0/en/>

- [4] D. Fernández, "Thymeleaf Documentation," Thymeleaf, 2023. [En línea]. Disponible: <https://www.thymeleaf.org/documentation.html>
- [5] Bootstrap Team, "Bootstrap 5.3 Documentation," 2024. [En línea]. Disponible: <https://getbootstrap.com/docs/5.3/>
- [6] E. Gamma, R. Helm, R. Johnson y J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.
- [7] Software Freedom Conservancy, "Git Documentation," Git, 2024. [En línea]. Disponible: <https://git-scm.com/doc>
- [8] K. Ramírez, J. Chan y D. Rayo, "Barbex — Sistema de Gestión de Barbería," repositorio GitHub, 2026. [En línea]. Disponible: <https://github.com/kennethDRE/ProyectoFinalDesarrolloAplicacionesWeb>